Lab No: 04

Student Name: Hania Nawab

Roll No: 17

Part 1: Single Circular Linked List

Task 1:	

Code:	void insertAtStart(string value) {
	node *temp = new node();
	temp->data = value;
	
	if (head == nullptr) {
	head = temp;
	temp->next = head;
	temp->prev = head;
	} else {
	node *last = head->prev;
	
	temp->next = head;
	temp->prev = last;
	
	last->next = temp;
	head->prev = temp;
	
	head = temp;
	}
	
	}
	
	void insertAtAnyPosition(string value,int pos)
	{
	node *temp =new node();
	temp->data = value;
	
	if (pos == 1)
	{

```
    insertAtStart(value);
    return;
  }

  node *tem = head;
  for(int i=0; i<pos-2; i++)
  {
    tem = tem->next;
  }
  temp->next = tem->next;
  tem->next = temp;
}

void insertAtEnd(string value)
{
  node *temp = new node();
  temp->data = value;

  if (head == nullptr)
  {
    head = temp;
    temp->next = head;
    temp->prev = head;
    return;

  }else{

    node *last = head->prev;

    last->next = temp;
    temp->prev = last;
    temp->next = head;
    head->prev = temp;
  }
}
```

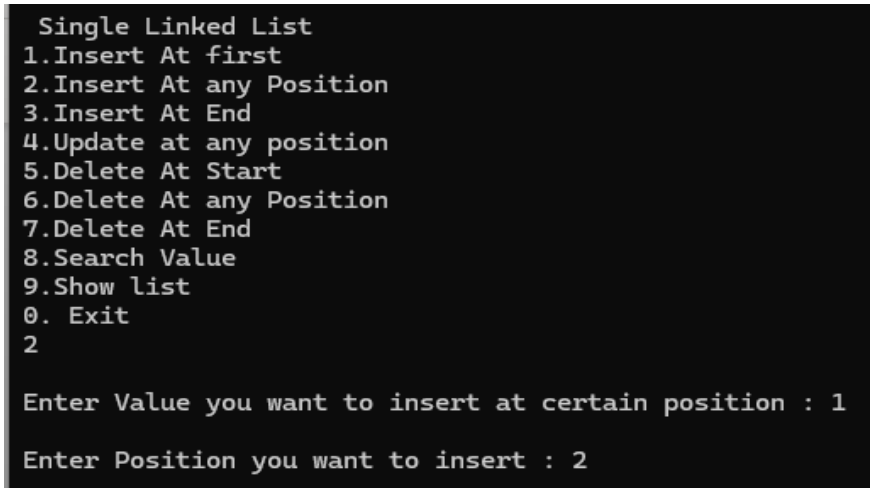
Screenshots:	

```
 Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
1

Enter Value you want to insert at first : 0
```

```
 Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
3

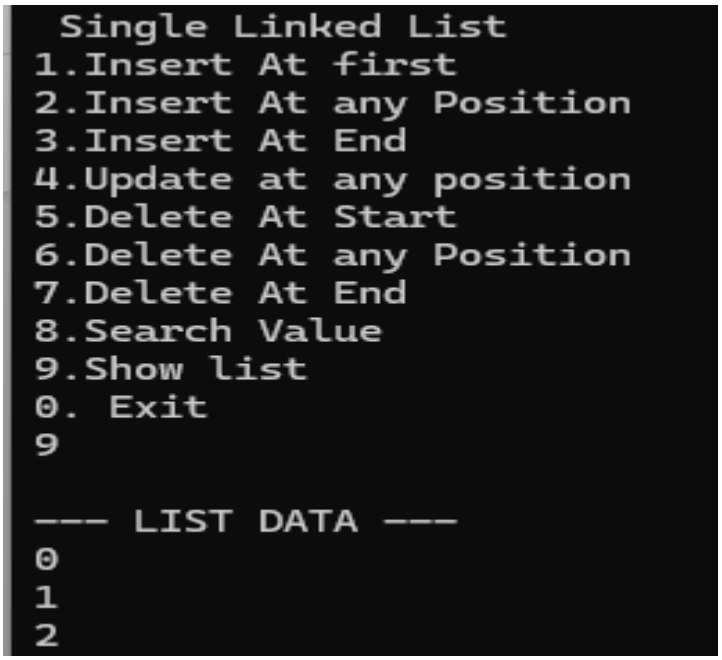
Enter Value you want to insert at end : 2
```



Task 2:

Code:

```
void showlist()
{
   node *temp = head;
   if (temp == nullptr)
   {
      cout << "List is empty!" << endl;
      return;
   }
   else {
   while(temp->next!= head)
   {
      cout<<(temp->data)<<endl;
      temp = temp->next;
   }  cout<<(temp->data)<<endl;
    return;
   }
}
```

Screenshot:	

Task 3:	

Code:	void DeleteAtStart()
	{
	if(head == nullptr)
	{
	cout<<"List is Empty"<<endl;
	return;
	}
	else if(head->next==head)
	{
	delete head;
	head = nullptr;
	return;
	}
	else{
	node *Todelet = head;
	node *last = head->prev;
	
	head = head->next;
	last->next = head;
	head->prev = last;
	
	delete Todelet;
	return;
	}
	}
	
	void DeleteAtAnyPosition(int pos)
	{
	if (pos == 1)
	{

	DeleteAtStart();

```
    return;
   }

else{
   node *prev = head;
   for(int i=0; i<pos-2; i++)
   {
     prev = prev->next;
   }
   node *Delet = prev->next;
   prev->next = Delet->next;
   delete Delet;
   return;
}
}

void DeleteAtEnd()
{
   if(head==nullptr)
   {
         cout<<"List is Emty"<<endl;
         return;
   }

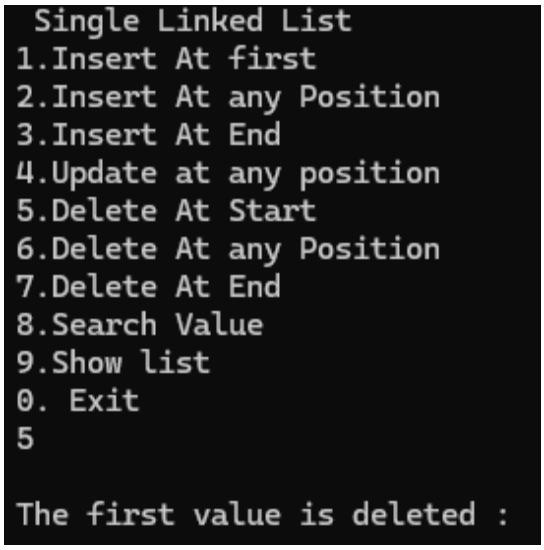
    if (head->next == head)
   {
      delete head;
      head=nullptr;
      return;

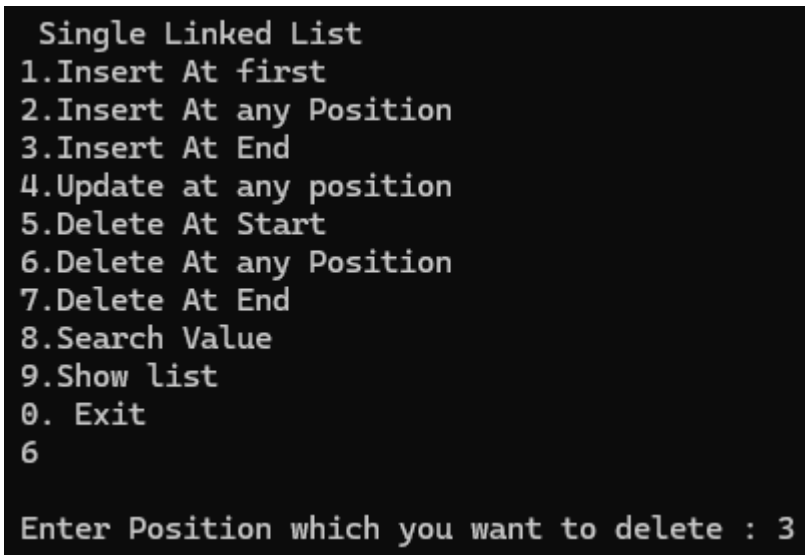
   }else{

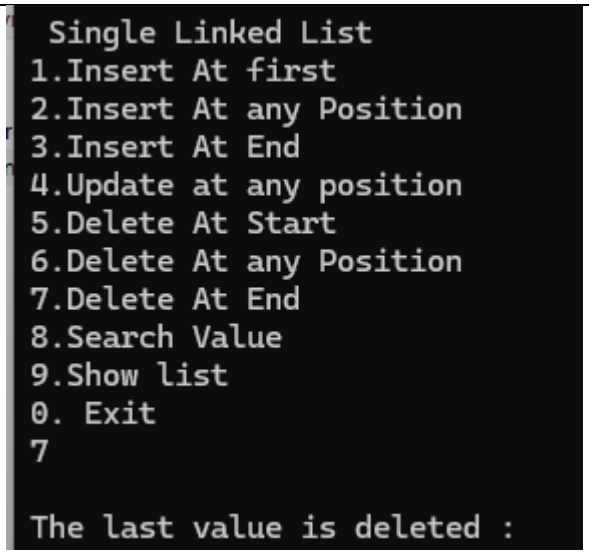
     node *prev = head;
     node *curr = head->next;
      while(curr->next != head){
         prev = curr;
         curr = curr->next;
      }
          prev->next = head;
         delete curr;
   }
}
```

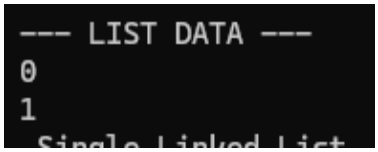
Screenshot:


--- LIST DATA ---
hania
0
1
2
aira


Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
5
The first value is deleted :


Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
6
Enter Position which you want to delete : 3





Task 4:	

Code:	void SearchValue(string value)
	{
	node *temp = head;
	int pos = 1;
	
	do{
	if(temp->data == value)
	{
	cout<<value<< " Exist at Positon : "<< pos<<endl;
	return;
	}
	temp = temp->next;
	pos++;
	}while(temp!=head);
	
	cout<<value<<" does not exist. "<<endl;
	}

Screenshot:	

```
 Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
8

Enter Value you want to Search in the list : 2

2 does not exist.
```

```
 Single Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
8

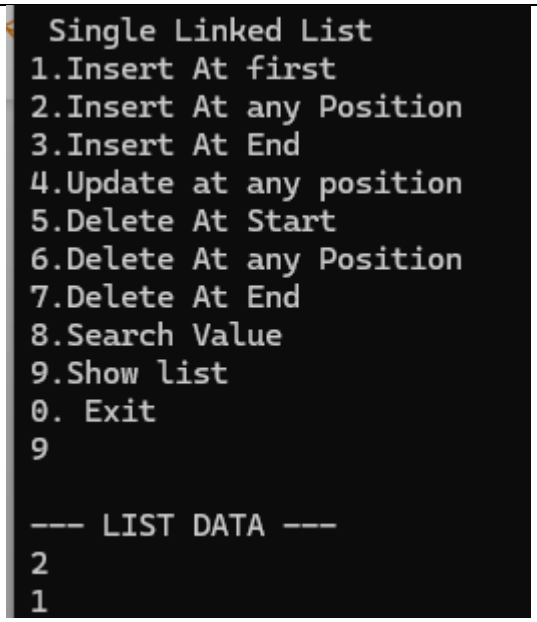
Enter Value you want to Search in the list : 1

1 Exist at Positon : 2
```

Task 5:	

Code:	void UpdateAtAnyPosition(string value,int pos)
	{
	node *tem = head;
	for(int i=0; i<pos-1; i++)
	{
	tem = tem->next;
	}
	tem->data = value;
	}

Screenshots :	--- LIST DATA --- 0 1 Single Linked List 1.Insert At first 2.Insert At any Position 3.Insert At End 4.Update at any position 5.Delete At Start 6.Delete At any Position 7.Delete At End 8.Search Value 9.Show list 0. Exit 4 Enter Value you want to update at certain position : 2 Enter Position you want to insert : 1

Part 2: Doubly Circular Linked List

Task 1:	

Code:	void insertAtStart(string value) {
	node *temp = new node();
	temp->data = value;
	
	if (head == nullptr) {
	head = temp;
	temp->next = head;
	temp->prev = head;
	} else {
	node *last = head->prev;
	
	temp->next = head;
	temp->prev = last;
	
	last->next = temp;
	head->prev = temp;
	
	head = temp;

```
    }
}

void insertAtAnyPosition(string
    value,int pos)
{
    if (pos == 1)
    {
      insertAtStart(value);
      return;
    }

    node *temp =new node();
    temp->data = value;

    node *tem = head;
    for(int i=0; i<pos-2; i++)
    {
       tem = tem->next;
    }
    temp->next = tem->next;
    temp->prev = tem;
    tem->next->prev = temp;
    tem->next = temp;
}

void insertAtEnd(string value)
{
    node *temp = new node();
    temp->data = value;

    if (head == nullptr)
    {
        head = temp;
        temp->next = head;
        temp->prev = head;
        return;
```

	}else{
	
	node *last = head->prev;
	
	last->next = temp;
	temp->prev = last;
	temp->next = head;
	head->prev = temp;
	}
	
	}

Screenshots:	

```
 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
1

Enter Value you want to insert at first : 1


 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
3

Enter Value you want to insert at end : 2


 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
2

Enter Value you want to insert at certain position : 3

Enter Position you want to insert : 2
```

Task 2:

Code:

```
void showlist()
{
   node *temp = head;
   if (temp == nullptr)
   {
      cout << "List is empty!" << endl;
      return;
   }
   else {

   do{
      cout<<(temp->data)<<endl;
      temp = temp->next;
   }  while(temp!= head);

   }
}
```

Screenshot:



Task 3:

Code:

```
   void DeleteAtStart()
   {
      if(head == nullptr)
      {
         cout<<"List is Empty"<<endl;
         return;
      }
      else if(head->next==head)
      {
```

```
        delete head;
        head = nullptr;
        return;
    }
    else{
        node *Todelet = head;
        node *last = head->prev;

        head = head->next;
        last->next = head;
        head->prev = last;

        delete Todelet;
        return;
    }
}

void DeleteAtAnyPosition(int pos)
{
    if (pos == 1)
    {
      DeleteAtStart();
      return;
    }

 else{
    node *pre = head;
    for(int i=0; i<pos-2; i++)
    {
       pre = pre->next;
    }
    node *Delet = pre->next;
    pre->next = Delet->next;
    Delet->next->prev =pre;
    delete Delet;
    return;
 }
}

void DeleteAtEnd()
{
    if(head==nullptr)
    {
        cout<<"List is Emty"<<endl;
        return;
    }

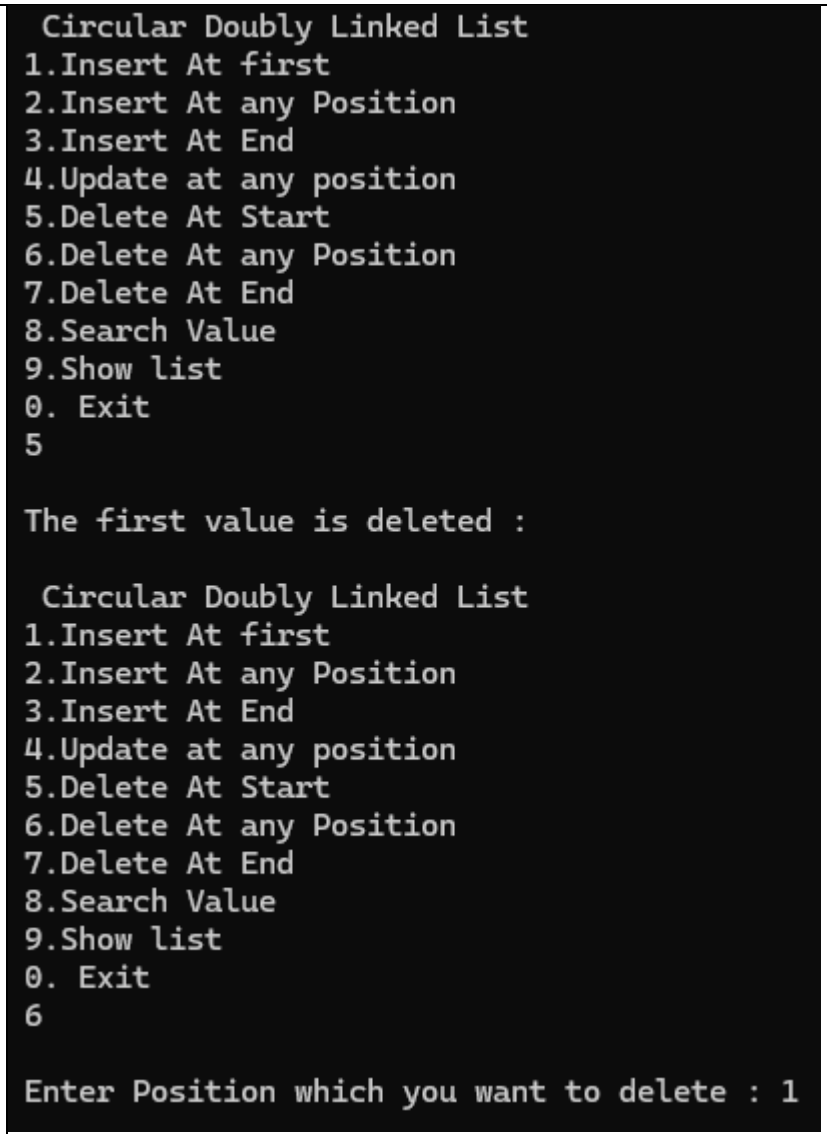
      if (head->next == head)
    {
        delete head;
        head=nullptr;
        return;
```

```
    }else{

      node *last = head->prev;
      node *secondlast = last->prev;

           secondlast->next = head;
           head->prev = secondlast;
          delete last;
    }
  }
```

Screenshot:



```
 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
7

The last value is deleted :
 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
9

--- LIST DATA ---
List is empty!
```

Task 4:

Code:

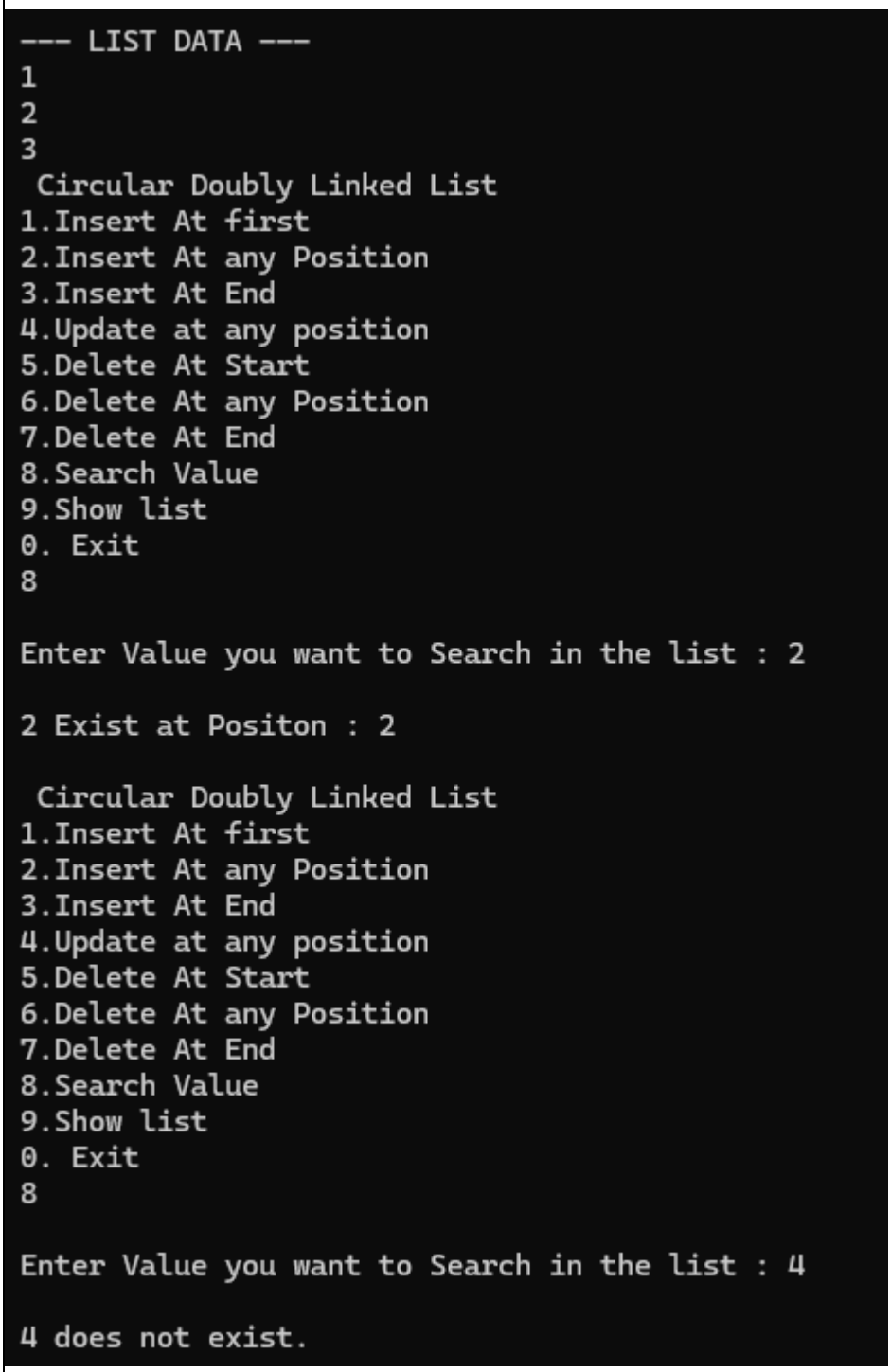
```
void SearchValue(string value)
{
   if(head == nullptr)
   {
      cout<<"List is Empty "<<endl;
   }

   node *temp = head;
   int pos = 1;

  do{
     if(temp->data == value)
   {
     cout<<value<< " Exist at Positon : "<< pos<<endl;
     return;
   }
   temp = temp->next;
   pos++;
  }while(temp!=head);
```

```
cout<<value<<" does not exist. "<<endl;
}
```

Screenshot:



Task 5:	

Code:	void UpdateAtAnyPosition(string value,int pos)
	{
	node *tem = head;
	for(int i=0; i<pos-1; i++)
	{
	tem = tem->next;
	}
	tem->data = value;
	}

Screenshots:

```
--- LIST DATA ---
1
2
3
 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
4

Enter Value you want to update at certain position : 4

Enter Position you want to insert : 2

 Circular Doubly Linked List
1.Insert At first
2.Insert At any Position
3.Insert At End
4.Update at any position
5.Delete At Start
6.Delete At any Position
7.Delete At End
8.Search Value
9.Show list
0. Exit
9

--- LIST DATA ---
1
4
3
```